

UNIVERSIDAD VERACRUZANA

FACULTAD DE INGENIERÍA ELÉCTRICA Y
ELECTRÓNICA

PROYECTO FINAL

MedAgenda

*Plataforma SaaS de Gestión Clínica.
Proyecto Ejecutivo Integral*



PROGRAMA EDUCATIVO

Ingeniería Informática

EXPERIENCIA EDUCATIVA

Evaluación de Proyectos / Diseño de Aplicaciones Web

DOCENTE

Lorena Nevero Arriola / Yuliana Berumen Díaz

ALUMNO

Madrigal Lagunes Orlando

Índice

1. Introducción	2
2. Descripción General	2
2.1. Propósito y Alcance	2
2.2. Objetivos	2
2.3. Requisitos y Funciones Principales	3
3. Diseño	4
3.1. Arquitectura Utilizada (Patrón MVC)	4
3.2. Diseño de la Base de Datos (Enfoque Objeto-Relacional)	4
3.3. Decisiones de Implementación	4
4. Diagramas UML	5
5. Interfaz Gráfica de Usuario (GUI)	8
6. Manual de Usuario	10
6.1. Acceso Principal y Registro de Usuarios	10
6.2. Aduana de Seguridad y Verificación	11
6.3. Paneles de Control (Dashboards)	11
6.4. Flujo Operativo del Paciente	11
6.5. Flujo Operativo del Médico	12
7. Conclusiones y Trabajo Futuro	14
7.1. Conclusiones	14
7.2. Trabajo Futuro	15
8. Anexos	16
8.1. Anexo A: Script de Base de Datos (DDL)	16
8.2. Anexo B: Diagrama de la Base de Datos	18

1 Introducción

La digitalización y el blindaje de los servicios de salud representan uno de los retos tecnológicos y normativos más críticos en la actualidad. El presente documento detalla el diseño, desarrollo e implementación de **MedAgenda**, una plataforma web SaaS (Software as a Service) orientada a la optimización operativa y seguridad de identidad en consultorios médicos independientes y policlínicas.

A diferencia de los sistemas de gestión tradicionales, MedAgenda no solo aborda la automatización de flujos comunes, sino que integra mecanismos avanzados de seguridad perimetral en el registro, verificación automatizada de credenciales profesionales, persistencia basada en el paradigma objeto-relacional y una interfaz reactiva enfocada en la experiencia de usuario (UX). El sistema se rige bajo el estándar de la arquitectura Java EE, garantizando un entorno robusto, escalable y seguro.

2 Descripción General

2.1 Propósito y Alcance

El propósito de MedAgenda es proveer a los profesionales de la salud una infraestructura tecnológica centralizada que mitigue la carga administrativa de sus consultorios. El alcance del proyecto comprende un ecosistema integral que abarca: la autenticación y control de acceso por roles, un buscador público de especialistas, el agendamiento dinámico de citas basado en horarios laborales, la consulta de un directorio clínico interactivo mediante componentes colapsables, el procesamiento de cobros con persistencia financiera, y un estricto filtro de seguridad que valida las cédulas profesionales antes de permitir el alta en la base de datos.

2.2 Objetivos

- **General:** Diseñar e implementar una aplicación web funcional y segura bajo el patrón arquitectónico MVC en Java, soportada por el motor de base de datos PostgreSQL, para la administración eficiente y fidedigna de servicios médicos.
- **Específicos:**
 - Implementar una aduana de seguridad en el flujo de registro que consulte un padrón oficial simulado para validar la existencia de la cédula y la identidad del profesionista.
 - Desarrollar una integración asíncrona con el servicio externo de *Resend API* para el envío de correos electrónicos transaccionales con plantillas HTML corporativas.
 - Diseñar un esquema de base de datos relacional interconectado que aproveche las características objeto-relacionales de PostgreSQL para estructurar los datos antropomórficos de los usuarios.
 - Optimizar la experiencia de usuario (UX) mediante técnicas de manipulación del historial del navegador para evitar la persistencia infinita de alertas visuales.

2.3 Requisitos y Funciones Principales

Requisitos Funcionales:

- **RF-01 Control de Roles:** El sistema debe segregar vistas, menús laterales y permisos de ejecución de Servlets según el rol del usuario autenticado (Doctor vs. Paciente).
- **RF-02 Aduana de Cédula (SEP):** El registro de un usuario tipo “Doctor” requiere obligatoriamente una cédula profesional de 8 dígitos. El sistema debe validar que la cédula exista en el Registro Nacional de Profesionistas y que el nombre ingresado coincida exactamente con el dictamen oficial para evitar la usurpación de identidad.
- **RF-03 Notificación de Verificación:** Tras un registro exitoso, el sistema debe disparar un token único y enviar un correo responsivo mediante un webhook externo para la activación de la cuenta.
- **RF-04 Directorio e Historial Clínico:** El médico debe disponer de una sección de lectura indexada que recopile a sus pacientes atendidos, desglosando de manera inmediata su motivo de consulta, diagnóstico emitido y tratamiento recetado en un formato interactivo de acordeón.
- **RF-05 Módulo Financiero:** El sistema debe permitir la liquidación de citas en estado “Realizada”, actualizando su estado a “Pagada” e insertando un registro en la bitácora financiera.

Requisitos No Funcionales:

- **RNF-01 Persistencia y Escalabilidad:** El almacenamiento de datos debe gestionarse a través de un pool de conexiones JDBC hacia PostgreSQL, asegurando la integridad referencial y transaccional.
- **RNF-02 Seguridad y Privacidad:** Las contraseñas de los usuarios deben encriptarse mediante algoritmos criptográficos antes de almacenarse. Las rutas protegidas de los Servlets deben desviar las peticiones al `index.jsp` si la sesión HTTP es nula.
- **RNF-03 Optimización de Memoria y UX:** Las alertas de confirmación en la interfaz del usuario deben contar con un temporizador automático de desvanecimiento y limpiar los parámetros de la URL de la barra de direcciones (`history.replaceState`) para evitar re-envíos duplicados al presionar F5.

3 Diseño

3.1 Arquitectura Utilizada (Patrón MVC)

El núcleo de MedAgenda se rige bajo el patrón Modelo-Vista-Controlador (MVC), logrando un desacoplamiento absoluto entre la lógica de presentación y las reglas de negocio:

- **Modelo (JavaBeans & DAOs):** Las entidades del sistema se mapean mediante POJOs encapsulados (ej. `Usuario.java`, `CedulaSEP.java`, `HistorialClinico.java`). La interacción con la base de datos se delega a las clases Data Access Object (`usuarioDAO.java`, `sepDAO.java`, `historialDAO.java`) que gestionan las consultas preparadas (`PreparedStatement`) contra el driver JDBC.
- **Vista (JSP, CSS, Bootstrap):** Capa encargada de renderizar los paneles de control. Integra lógicas embebidas de Java en los JSP para desplegar información de sesión, componentes dinámicos de Bootstrap y animaciones CSS de flotación para el área comercial.
- **Controlador (Servlets Java):** Componentes web como `usuarioServlet.java` actúan como la aduana de control de peticiones HTTP (GET/POST). Atrapan los parámetros del formulario, invocan los filtros de validación de los DAOs, coordinan los servicios de mensajería asíncronos y redirigen el flujo hacia los destinos correspondientes.

3.2 Diseño de la Base de Datos (Enfoque Objeto-Relacional)

La persistencia del software se soporta sobre **PostgreSQL**. Con el objetivo de optimizar la reutilización de estructuras y estandarizar el almacenamiento de la información humana, se diseñó e implementó un tipo de dato compuesto personalizado denominado **PersonaNombre**, el cual agrupa de forma nativa los campos: `nombrepila`, `paterno` y `materno`.

Esta decisión de diseño impacta directamente a las tablas `doctor` y `sep_cedula`. Debido a que JDBC lee los tipos compuestos como cadenas tabuladas crudas, la extracción de datos en la capa DAO se solucionó aplicando técnicas de desestructuración directamente en el motor SQL mediante la sintaxis de campos compuestos `columna.atributo AS alias`, permitiendo una transferencia limpia de información hacia las propiedades planas del Modelo en Java.

3.3 Decisiones de Implementación

El ciclo de vida del software es gestionado formalmente por **Maven**, centralizando las dependencias del driver de PostgreSQL y la integración del SDK oficial de **Resend** para mitigar la latencia y fallas de entrega comunes del protocolo SMTP tradicional.

En la capa de frontend, la fluidez visual del sistema se implementó inyectando scripts nativos al evento `DOMContentLoaded`. Estos scripts automatizan el ciclo de vida de las alertas mediante transiciones fluidas de opacidad de CSS y remueven los parámetros del *query string* de la URL del navegador en secreto, elevando los estándares de usabilidad de la plataforma.

4 Diagramas UML

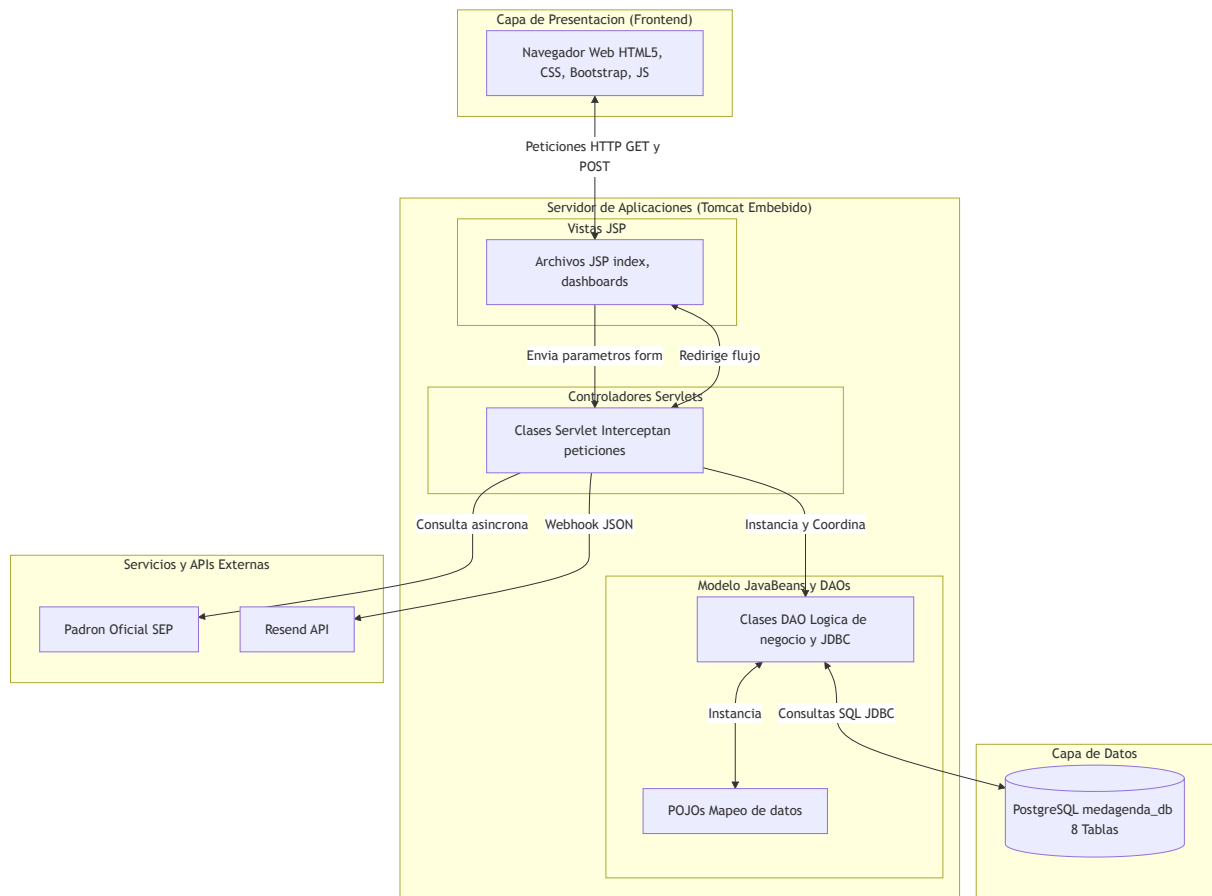


Figura 1: Arquitectura General del Proyecto

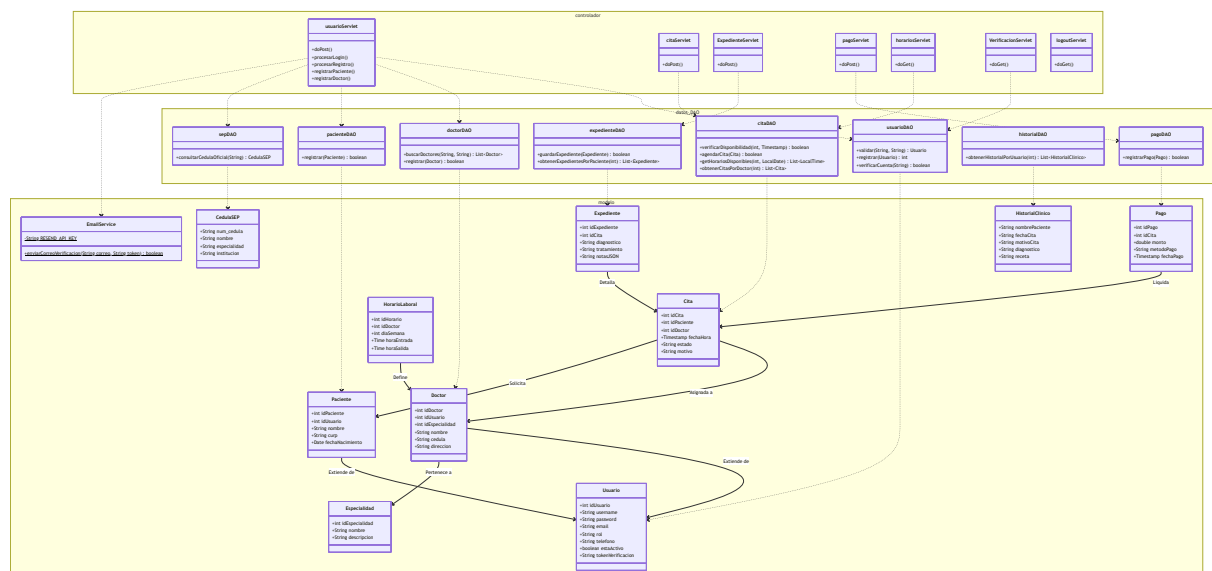


Figura 2: Diagrama de Clases del Proyecto

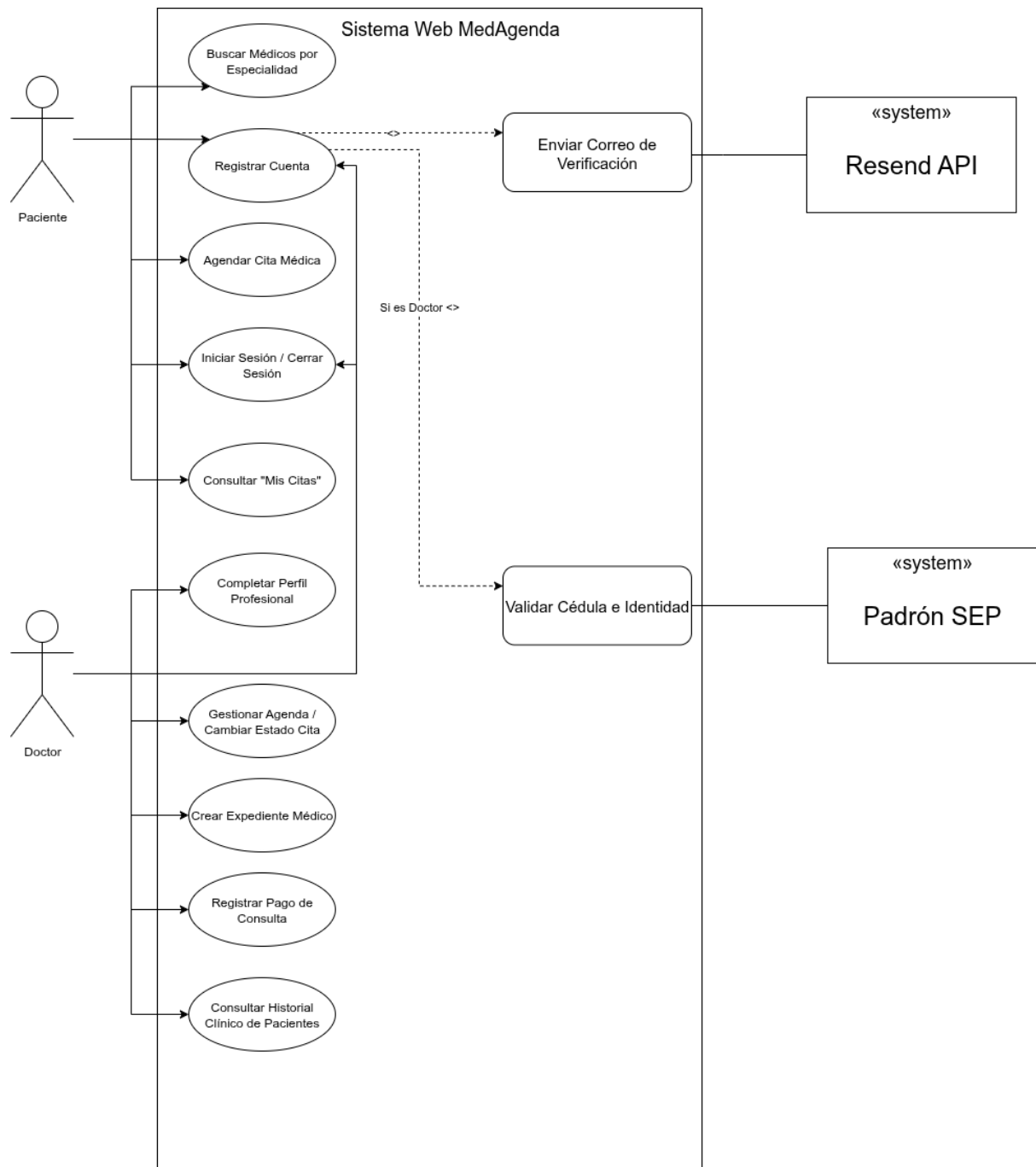


Figura 3: Diagrama de Casos de Uso

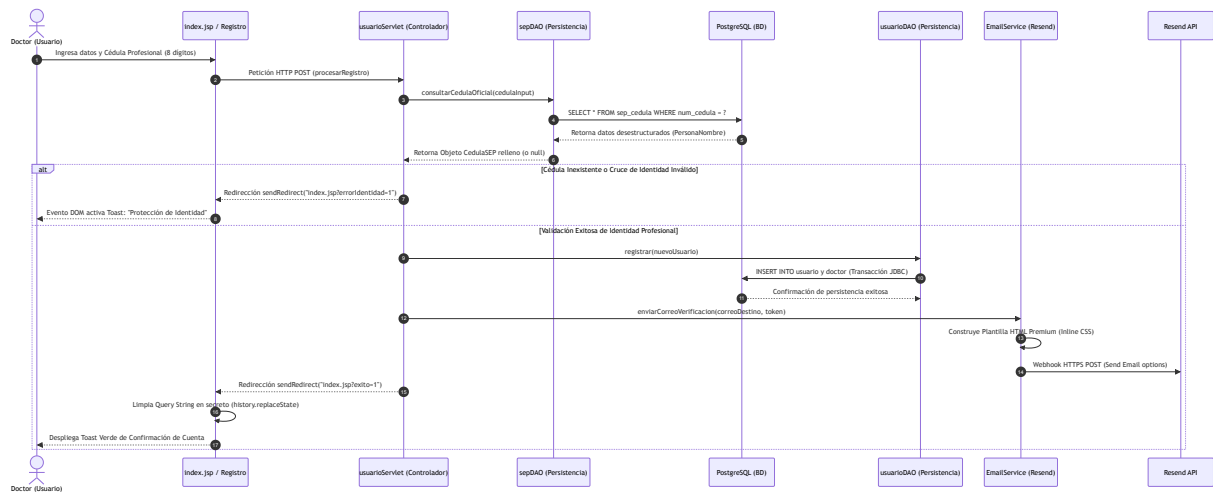


Figura 4: Diagrama de Secuencia: Validación de Cédula Profesional

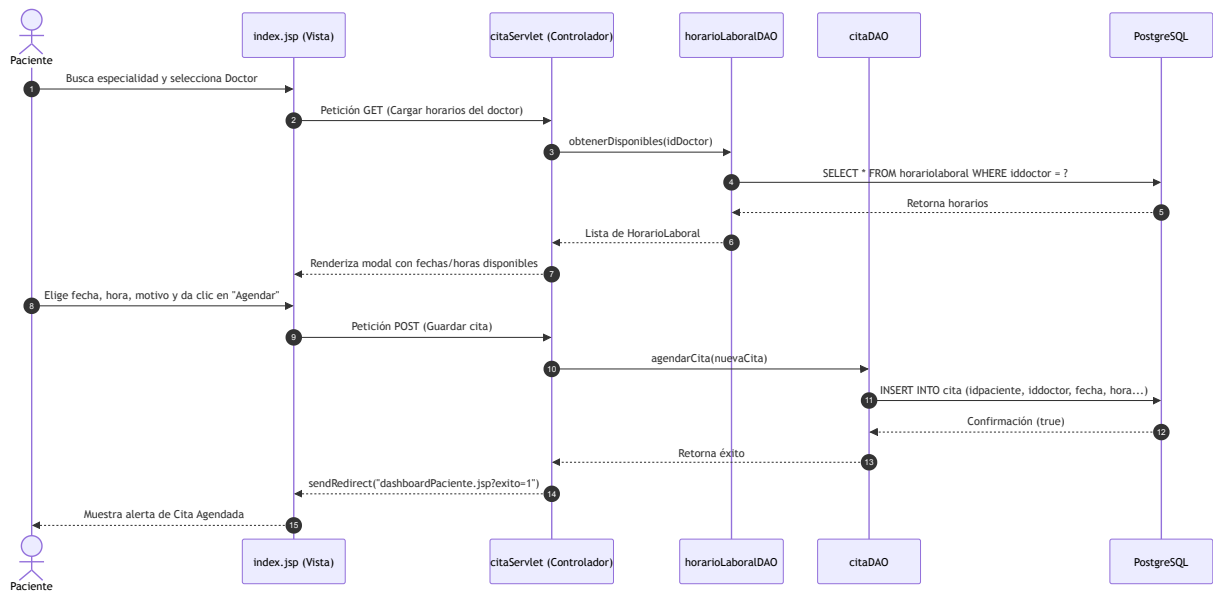


Figura 5: Diagrama de Secuencia: Agendado de Cita

5 Interfaz Gráfica de Usuario (GUI)

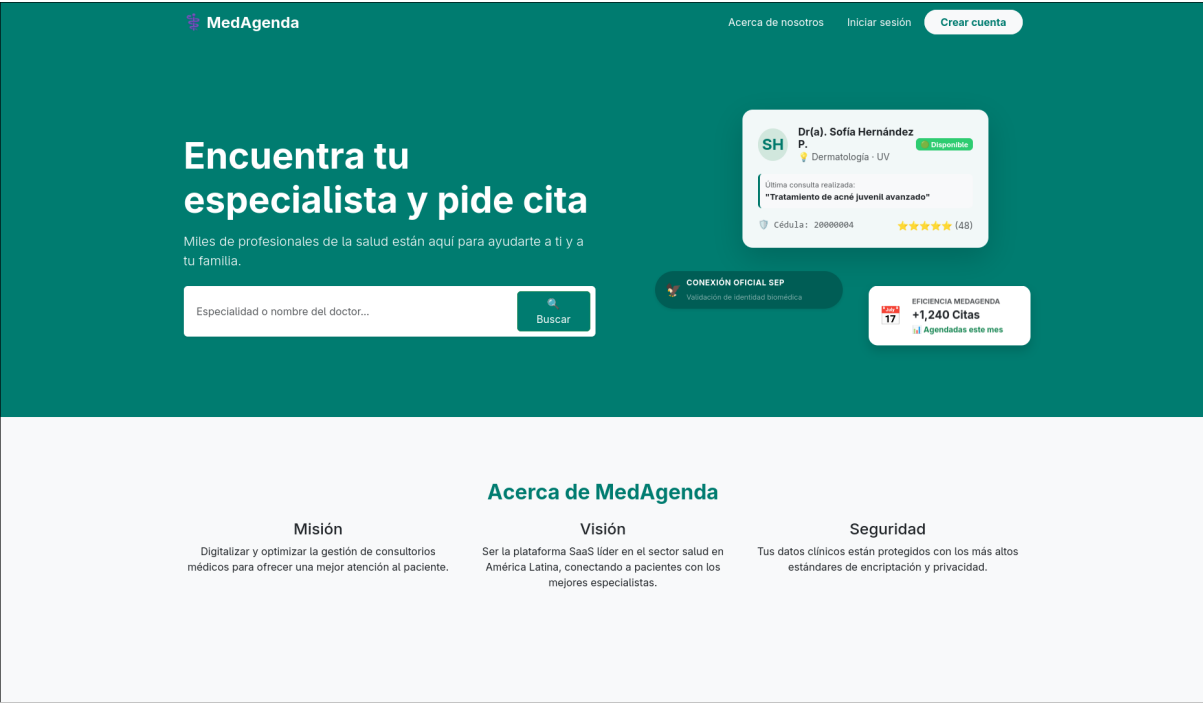
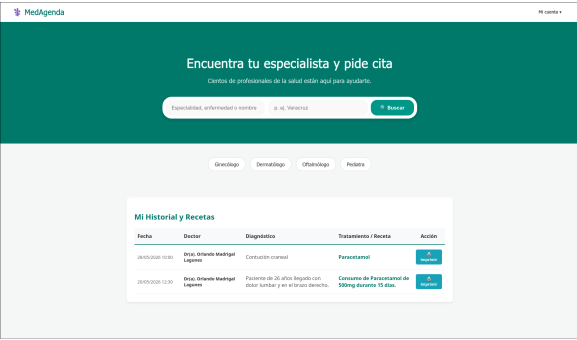
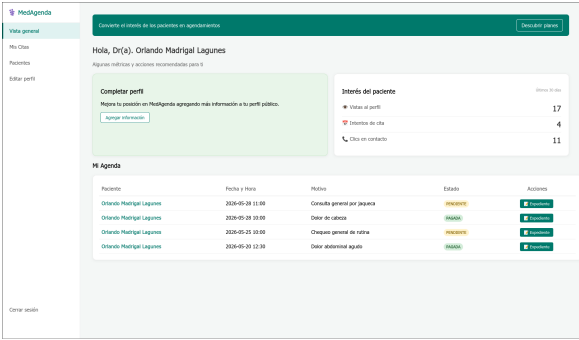


Figura 6: Página Principal de la Aplicación (previa al Login)

Dashboards: Correspondientes al Tipo de Usuario



Dashboard para Pacientes



Dashboard para Doctores

Páginas de Eventos Principales por Tipo de Usuario

The screenshot shows a web form titled "Agendar Cita". It contains the following elements:

- 1. Seleccione el día de la consulta:** A date picker set to "14 / 04 / 2024".
- 2. Horarios disponibles para este día:** Four buttons labeled "15:00", "16:00", "17:00", and "18:00".
- Motivo de la consulta:** A text input field.
- Confirmar Cita:** A green button at the bottom.

Página de Agendado de Citas para Pacientes

The screenshot shows a web form titled "Nueva Nota Médica / Expediente" within a "MediAgenda - Panel Médico" interface. It contains the following elements:

- U. Diagnóstico Clínico:** A text input field with the placeholder "Escriba el diagnóstico detallado del paciente...".
- de Tratamiento y Prescripción:** A text input field with the placeholder "Indica los medicamentos, dosis y duración del tratamiento...".
- Datos Adicionales (Estructura Inteligente):** A section with two sub-inputs:
 - Alergias Reportadas:** A text input field.
 - Ex. Periclitina, marfeno (dejar vacío si no presenta):** A text input field.
- 4. Guardar:** A button on the left.
- U. Guardar Expediente:** A green button on the right.

Página de Generación de Expedientes Clínicos para Doctores

6 Manual de Usuario

El presente manual detalla de forma visual y estructurada el flujo operativo de **MedAgenda**, guiando paso a paso la interacción de los perfiles de Paciente y Médico con el sistema.

6.1 Acceso Principal y Registro de Usuarios

El acceso a la plataforma se realiza a través de la página principal, la cual presenta un buscador público de especialistas y los accesos directos para iniciar sesión o crear una cuenta nueva.

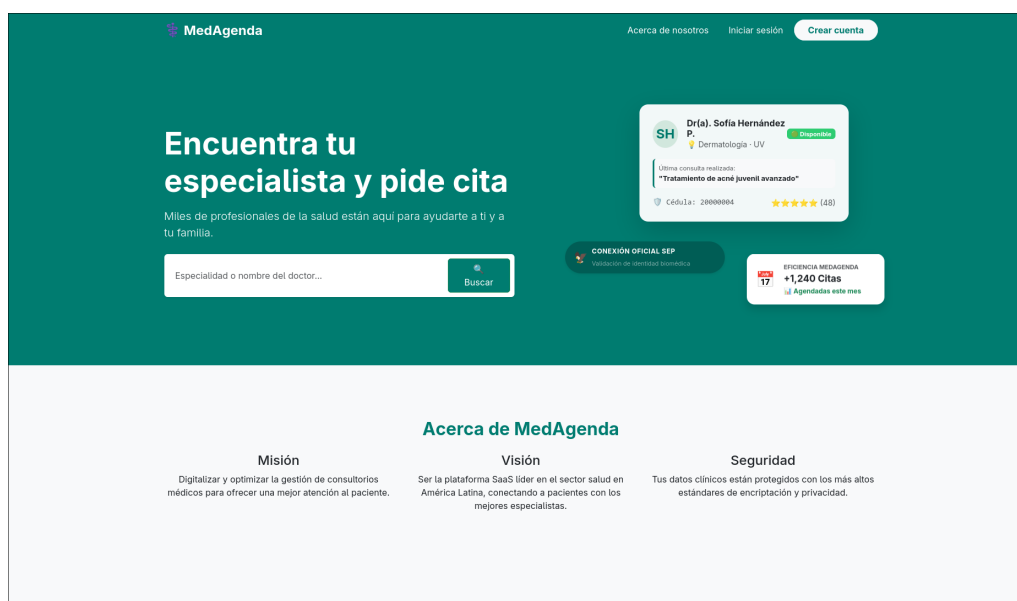


Figura 7: Landing page de MedAgenda con opciones de búsqueda, inicio de sesión y registro.

Para acceder a las funciones del sistema, los usuarios deben registrarse seleccionando su rol. El formulario se adapta dinámicamente mediante JavaScript: el paciente proporciona datos de filiación generales, mientras que el médico debe ingresar obligatoriamente su Cédula Profesional y especialidad para someterse a evaluación.

Formularios Dinámicos de Registro por Rol

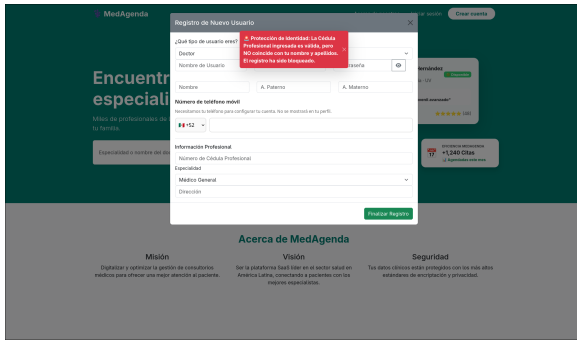
Registro de Paciente

Registro de Doctor (Requiere Cédula)

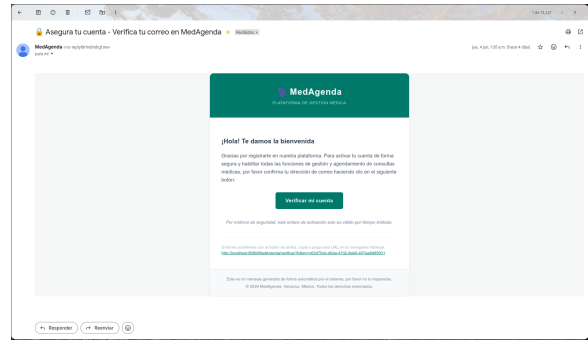
6.2 Aduana de Seguridad y Verificación

El sistema cuenta con mecanismos estrictos para proteger la identidad profesional. Si los datos del médico no coinciden con el registro del padrón oficial simulado, el sistema bloquea el alta y despliega una alerta crítica mediante un *Toast*. Si el registro supera este filtro, la cuenta se crea inactiva y se despacha un token de activación al correo del solicitante.

Mecanismos de Validación y Seguridad



Bloqueo por Usurpación de Identidad

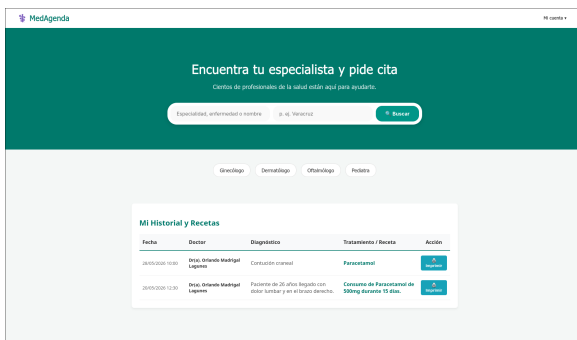


Token de Verificación vía Email

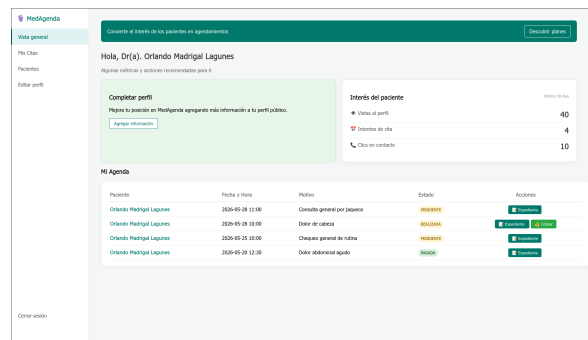
6.3 Paneles de Control (Dashboards)

Una vez verificados, los usuarios acceden a su respectivo *Dashboard*, donde las funcionalidades, menús y métricas están segregadas por el control de acceso que define el patrón MVC.

Interfaces Principales (Vistas Protegidas)



Panel de Gestión del Paciente



Panel Operativo del Médico

6.4 Flujo Operativo del Paciente

Desde su panel, el paciente puede localizar a su médico y apartar un espacio en la agenda. El sistema calcula en tiempo real los horarios disponibles cruzando la tabla de horarios laborales y las citas previas para evitar colisiones.

Agendar Cita

1. Selecciona el día de tu consulta:

16 / 06 / 2020

2. Horarios disponibles para este día:

09:00 10:00 11:00 12:00 13:00

Motivo de la consulta:

Confirmar Cita

Figura 8: Modal interactivo para agendar citas verificando disponibilidad en tiempo real.

6.5 Flujo Operativo del Médico

La gestión clínica del doctor se divide en dos fases: el registro del encuentro actual y la revisión de antecedentes. Al atender a un paciente, el médico ingresa su valoración clínica, cambiando automáticamente el estado de la cita a *REALIZADA*.

MedAgenda - Panel Médico

Nueva Nota Médica / Expediente

Diagnóstico Clínico

Escribe el diagnóstico detallado del paciente...

Tratamiento y Prescripción

Indica los medicamentos, dosis y duración del tratamiento...

Datos Adicionales (Estructura Inteligente)

Alergias Reportadas

Ej: Penicilina, mariscos (dejar vacío si no presenta)

Volver Guardar Expediente

Figura 9: Módulo de redacción para el diagnóstico y receta clínica (Expediente).

Para agilizar la toma de decisiones, el médico cuenta con un directorio de pacientes en formato de acordeón colapsable. Esta interfaz permite la lectura inmediata de diagnósticos y recetas pasadas sin necesidad de navegar entre múltiples ventanas, mejorando la usabilidad.

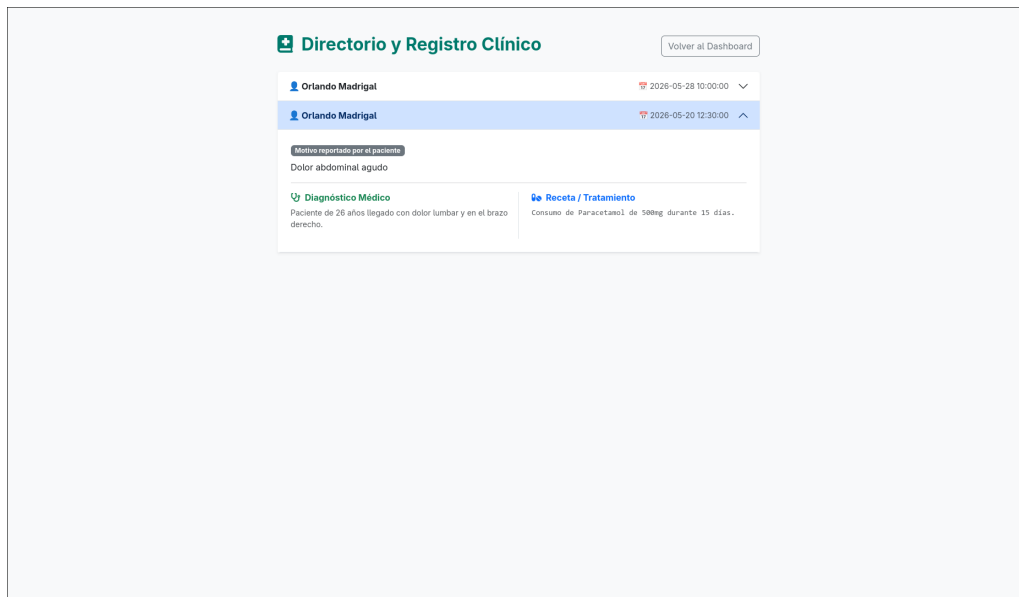


Figura 10: Directorio interactivo de pacientes con historial clínico colapsable.

7 Conclusiones y Trabajo Futuro

7.1 Conclusiones

El desarrollo de **MedAgenda** ha sido una de las experiencias más desafiantes y enriquecedoras de la carrera. Pasar de hacer pequeños programas durante las EE a maquetar una plataforma SaaS completa, donde la capa de presentación en los JSP es sumamente importante ya que será lo que se le presentará al usuario final, pero sin descuidar en lo absoluto el back-end ya que se encargará de que, además de verse bien, todo funcione correctamente, usando directamente consultas relacionales complejas y filtros de seguridad, te cambia por completo la perspectiva como futuro Ingeniero Informático. Este proyecto me obligó a pensar no solo en que el código sea funcional, sino en cómo estructurarlo para que sea escalable, seguro y fácil de usar.

Como en todo proyecto real, el camino no estuvo libre de tropiezos. Durante las fases de integración surgieron problemáticas técnicas críticas que me obligaron a profundizar en la arquitectura:

- **El manejo de sesiones y el clásico `NullPointerException`:** Uno de los dolores de cabeza más comunes me ocurrió al implementar el Directorio de Pacientes al intentar extraer datos de sesión asumiendo que existían ya doctores registrados (no contemplé inicialmente el hacer un insert con datos dummy) me generaba constantemente colapsos en el servidor de Tomcat. La solución no fue saturar el Servlet de inicio de sesión con más variables, sino rediseñar la lógica del `historialDAO` para que realizara un JOIN transparente hacia la tabla de médicos utilizando el `idUsuario`, el cual ya vivía de forma segura en la sesión global.
- **La rigidez del tipado en la Base de Datos:** El trabajar con características objeto-relacionales en PostgreSQL (y directamente con PostgreSQL, el cual nunca había utilizado) fue una experiencia, ya que consideraba que sería mucho más complejo de lo que fue, al tener una idea errónea de lo que sería PostgreSQL a lo que en realidad es. Darme cuenta que usando características como el tipo compuesto `personanombre`, fue una excelente decisión de diseño pero también un reto al mapearlo en Java. Además, pequeños desfases tipográficos entre el código y la base de datos (como intentar leer una columna como `fecha` cuando en PostgreSQL la habíamos nombrado `fechahora`, o el uso de mayúsculas y minúsculas al mandar a llamar las diferentes tablas de la base de datos) hacían que el driver JDBC arrojara excepciones de mapeo. Esto se solucionó homogeneizando los JavaBeans y forzando alias semánticos (`AS fecha`) directamente en las sentencias preparadas de SQL.

A nivel técnico, el descubrir que podía tener un dominio gratuito justamente en el periodo en que necesitaba implementarlo para el proyecto, e integrar la API de *Resend* fue un acierto enorme; me evitó lidiar con las limitaciones y la latencia del protocolo SMTP tradicional de JavaMail, permitiendo que la verificación de cuentas se sienta como un producto comercial moderno. Asimismo, detalles como limpiar la barra de direcciones con `history.replaceState` tras un envío de formulario demostraron que se puede cuidar la integridad del pool de conexiones controlando el comportamiento del navegador.

7.2 Trabajo Futuro

MedAgenda se entrega como un Producto Mínimo Viable (MVP) completamente funcional, estable y blindado. Sin embargo, para una segunda fase de desarrollo se tienen proyectadas las siguientes mejoras:

- **Persistencia avanzada de expedientes:** Migrar el almacenamiento de las recetas y notas médicas hacia el tipo de dato nativo `JSONB` de PostgreSQL para permitir campos dinámicos (como alergias múltiples o antecedentes familiares) sin alterar el esquema relacional.
- **Módulo de Configuración de Perfil Extendido:** Activar las rutas del formulario estético de actualización de perfil para que los médicos puedan editar sus datos de contacto del consultorio y costos de consulta directamente en la base de datos.
- **Pasarela de Pagos:** Integrar una API financiera (como Stripe o PayPal) para que el estado de la cita pase de *REALIZADA* a *PAGADA* mediante una transacción digital segura antes de la consulta.
- *Posible implementación personalizada para consultorios:* En el momento de lograr comercializar la aplicación, poder ofrecer planes personalizados según las necesidades específicas de cada clínica, principalmente en lo visual (para que se diferencie del resto) como en funcionalidades que tenga cada una y que requiera para su correcto funcionamiento.

En conclusión, este proyecto demostró principalmente que el patrón MVC no es solo teoría; es la base para construir software limpio, donde el backend es una base esencial e indispensable y el frontend es un entorno amigable para el usuario. Con él aprendí que, a pesar de tener actualmente muchas herramientas que facilitan el trabajo, nada se compara a tener un entendimiento total sobre todo el trasfondo de la aplicación, en cómo funciona cada módulo, qué hace cada sección, entre otras cosas; enseñanzas que, de haber utilizado parcial o en su totalidad frameworks y no haber realizado todo .^a manócomo fue el caso, seguramente me habría sido más complicado obtener.

8 Anexos

8.1 Anexo A: Script de Base de Datos (DDL)

El siguiente código SQL corresponde al lenguaje de definición de datos (DDL) ejecutado en PostgreSQL. Se incluye la creación del tipo compuesto orientado a objetos y la normalización de las 8 tablas principales junto con el catálogo de validación de la SEP.

```
1 CREATE TYPE personanombre AS (  
2     nombrepila VARCHAR(50),  
3     paterno VARCHAR(50),  
4     materno VARCHAR(50)  
5 );  
6  
7 CREATE TABLE usuario (  
8     idusuario SERIAL PRIMARY KEY,  
9     username VARCHAR(50) NOT NULL UNIQUE,  
10    rol VARCHAR(20) NOT NULL CHECK (rol IN ('ADMIN', 'DOCTOR', '  
11        ASISTENTE', 'PACIENTE')),  
12    estaactivo BOOLEAN DEFAULT true,  
13    email TEXT NOT NULL UNIQUE,  
14    password VARCHAR(80),  
15    telefono VARCHAR(15) NOT NULL CHECK (telefono ~ '^([0-9]+)$'),  
16    token_verificacion VARCHAR(255)  
17 );  
18  
19 CREATE TABLE especialidad (  
20     idespecialidad SERIAL PRIMARY KEY,  
21     nombre personanombre,  
22     descripcion TEXT  
23 );  
24  
25 CREATE TABLE paciente (  
26     idpaciente SERIAL PRIMARY KEY,  
27     curp VARCHAR(18) UNIQUE,  
28     nombre personanombre,  
29     fechanacimiento DATE,  
30     idusuario INTEGER UNIQUE REFERENCES usuario(idusuario)  
31 );  
32  
33 CREATE TABLE doctor (  
34     iddoctor SERIAL PRIMARY KEY,  
35     idusuario INTEGER UNIQUE REFERENCES usuario(idusuario),  
36     idespecialidad INTEGER REFERENCES especialidad(idespecialidad),  
37     nombre personanombre,  
38     cedula VARCHAR(20) NOT NULL UNIQUE,  
39     direccion VARCHAR(255)  
40 );  
41  
42 CREATE TABLE cita (  
43     idcita SERIAL PRIMARY KEY,  
44     idpaciente INTEGER REFERENCES paciente(idpaciente),  
45     iddoctor INTEGER REFERENCES doctor(iddoctor),  
46     fechahora TIMESTAMP WITHOUT TIME ZONE NOT NULL,  
47     motivo VARCHAR(255),
```

```

47     estado VARCHAR(20) DEFAULT 'PENDIENTE' CHECK (estado IN ('PENDIENTE
48         ', 'REALIZADA', 'CANCELADA', 'PAGADA'))
49 );
50 CREATE TABLE horariolaboral (
51     idhorario SERIAL PRIMARY KEY,
52     iddoctor INTEGER REFERENCES doctor(iddoctor),
53     diasemana VARCHAR(15) NOT NULL,
54     horaentrada TIME WITHOUT TIME ZONE NOT NULL,
55     horasalida TIME WITHOUT TIME ZONE NOT NULL
56 );
57
58 CREATE TABLE expediente (
59     idexpediente SERIAL PRIMARY KEY,
60     idcita INTEGER UNIQUE REFERENCES cita(idcita),
61     diagnostico TEXT,
62     tratamiento TEXT,
63     notasjson JSONB
64 );
65
66 CREATE TABLE pago (
67     idpago SERIAL PRIMARY KEY,
68     idcita INTEGER UNIQUE REFERENCES cita(idcita),
69     monto NUMERIC(10,2) NOT NULL,
70     metodopago VARCHAR(50),
71     fechapago TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
72 );
73
74 CREATE TABLE sep_cedula (
75     num_cedula VARCHAR(10) PRIMARY KEY,
76     nombre personanombre,
77     especialidad VARCHAR(100),
78     institucion VARCHAR(100)
79 );

```

8.2 Anexo B: Diagrama de la Base de Datos

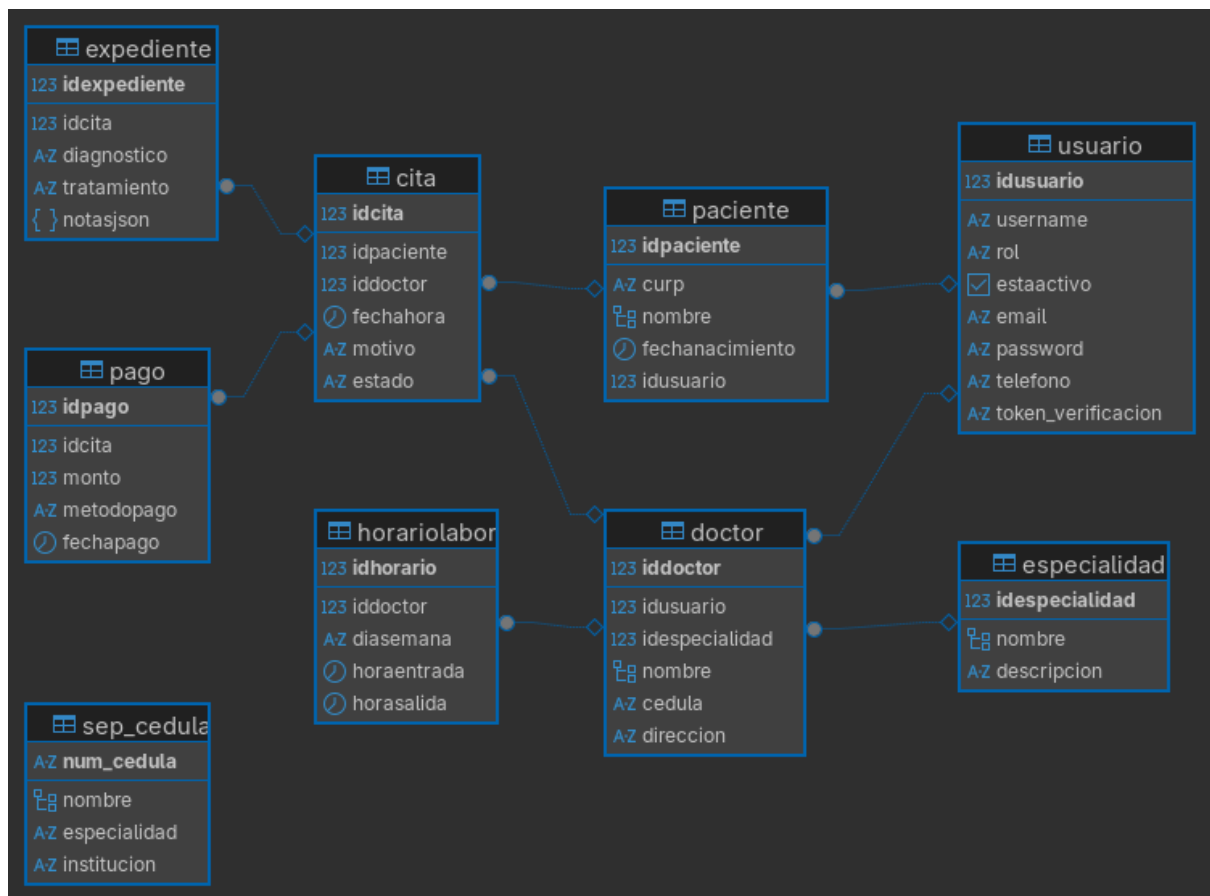


Figura 11: Diagrama de la Base de Datos